Module 4: System Coding

4.1 Code

4.1.1 Frontend Code (Next.js / TypeScript)

Synthesis Dashboard Component

File: `frontend/src/app/_components/synthesis-dashboard.tsx`

```
"use client";

import { useState, useCallback, useMemo } from "react";
import { useSession } from "next-auth/react";
import { api } from "~/trpc/react";

// Valid characters for synthesis
const VALID_CHARS = new Set([
  " ", "!", '"', "#", "'", "(", ")", ",", "-", ".",
  "0", "1", "2", "3", "4", "5", "6", "7", "8", "9",
  ":", ";", "?",
  "A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M",
  "N", "O", "P", "R", "S", "T", "U", "V", "W", "Y",
  "a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k", "l", "m",
  "n", "o", "p", "q", "r", "s", "t", "u", "v", "w", "x", "y", "z",
  "\n",
]);

const MAX_CHARS_PER_LINE = 75;
const MAX_LINES = 20;

export function SynthesisDashboard() {
  const { data: session } = useSession();
  const [text, setText] = useState("Hello World!");
  const [style, setStyle] = useState(9);
  const [bias, setBias] = useState(0.75);
  const [strokeColor, setStrokeColor] = useState("#000000");
  const [strokeWidth, setStrokeWidth] = useState(2);
  const [result, setResult] = useState(null);
  const [error, setError] = useState(null);

  const generateMutation = api.synthesis.generate.useMutation({
    onSuccess: (data) => {
      setResult(data);
      setError(null);
    },
    onError: (err) => {
      setError(err.message);
```

```
      setResult(null);
    },
  });

  const handleGenerate = () => {
    generateMutation.mutate({
      text,
      style,
      bias,
      strokeColor,
      strokeWidth,
    });
  };

  return (
    <div className="space-y-6">
      <textarea
        value={text}
        onChange={(e) => setText(e.target.value)}
        placeholder="Enter text to convert to handwriting..."
        className="w-full h-32 p-3 border rounded"
      />

      <div className="flex gap-4">
        <select value={style} onChange={(e) => setStyle(Number(e.target.value
))}>
          {[...Array(13)].map((_, i) => (
            <option key={i} value={i}>Style {i}</option>
          ))}
        </select>

        <input
          type="range"
          min="0" max="1.5" step="0.05"
          value={bias}
          onChange={(e) => setBias(Number(e.target.value))}
        />
      </div>

      <button onClick={handleGenerate} disabled={generateMutation.isPending}>
        {generateMutation.isPending ? "Generating..." : "Generate Handwriting
"}
      </button>

      {result && (
        <div dangerouslySetInnerHTML={{ __html: result.svgRaw }} />
      )}
    </div>
```

```
  );
}
```

tRPC Synthesis Router

File: `frontend/src/server/api/routers/synthesis.ts`

```
import { z } from "zod";
import { TRPCError } from "@trpc/server";
import { createTRPCRouter, protectedProcedure, publicProcedure } from "~/serv
er/api/trpc";

const API_URL = process.env.SYNTHESIS_API_URL;

export const synthesisRouter = createTRPCRouter({
  // Health check
  health: publicProcedure.query(async () => {
    const response = await fetch(`${API_URL}/health`);
    return await response.json();
  }),

  // Generate handwriting
  generate: protectedProcedure
    .input(z.object({
      text: z.string().min(1).max(1600),
      style: z.number().min(0).max(12).default(9),
      bias: z.number().min(0).max(1.5).default(0.75),
      strokeColor: z.string().default("black"),
      strokeWidth: z.number().min(1).max(5).default(2),
    }))
    .mutation(async ({ ctx, input }) => {
      const userId = ctx.session.user.id;

      // Check credits
      const user = await ctx.db.user.findUnique({
        where: { id: userId },
        select: { credits: true },
      });

      if (!user || user.credits < 1) {
        throw new TRPCError({
          code: "FORBIDDEN",
          message: "Insufficient credits",
        });
      }

      // Call synthesis API
      const response = await fetch(`${API_URL}/synthesize`, {
        method: "POST",
        headers: { "Content-Type": "application/json" },
```

```
      body: JSON.stringify({
        text: input.text,
        style: input.style,
        bias: input.bias,
        stroke_color: input.strokeColor,
        stroke_width: input.strokeWidth,
      }),
    });

    if (!response.ok) {
      throw new TRPCError({
        code: "INTERNAL_SERVER_ERROR",
        message: "Synthesis failed",
      });
    }

    const data = await response.json();

    // Deduct credit
    await ctx.db.user.update({
      where: { id: userId },
      data: { credits: { decrement: 1 } },
    });

    return {
      svg: data.svg,
      svgRaw: data.svg_raw,
      linesCount: data.lines_count,
      charactersCount: data.characters_count,
    };
  }),

// Save generation to gallery
saveGeneration: protectedProcedure
  .input(z.object({
    text: z.string(),
    style: z.number(),
    bias: z.number(),
    fileUrl: z.string().url(),
    fileKey: z.string(),
  }))
  .mutation(async ({ ctx, input }) => {
    return await ctx.db.savedGeneration.create({
      data: {
        userId: ctx.session.user.id,
        text: input.text,
        style: input.style,
        bias: input.bias,
        fileUrl: input.fileUrl,
```

```
        fileKey: input.fileKey,
        status: "COMPLETED",
      },
    });
  }),

// Get gallery
getGallery: protectedProcedure
  .input(z.object({
    limit: z.number().min(1).max(50).default(20),
    cursor: z.string().optional(),
  }))
  .query(async ({ ctx, input }) => {
    const items = await ctx.db.savedGeneration.findMany({
      where: { userId: ctx.session.user.id },
      take: input.limit + 1,
      cursor: input.cursor ? { id: input.cursor } : undefined,
      orderBy: { createdAt: "desc" },
    });

    let nextCursor = undefined;
    if (items.length > input.limit) {
      nextCursor = items.pop()!.id;
    }

    return { items, nextCursor };
  }),

// Make realistic
makeRealistic: protectedProcedure
  .input(z.object({
    generationId: z.string(),
    paperType: z.string().default("white"),
    inkType: z.string().default("ballpoint"),
    wearLevel: z.number().min(0).max(1).default(0.3),
  }))
  .mutation(async ({ ctx, input }) => {
    const generation = await ctx.db.savedGeneration.findFirst({
      where: { id: input.generationId, userId: ctx.session.user.id },
    });

    if (!generation) {
      throw new TRPCError({ code: "NOT_FOUND" });
    }

    const response = await fetch(`${API_URL}/process/realistic`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
```

```
          svg_content: generation.svgContent,
          paper_type: input.paperType,
          ink_type: input.inkType,
          wear_level: input.wearLevel,
        }),
      });

      const data = await response.json();

      return await ctx.db.savedGeneration.update({
        where: { id: input.generationId },
        data: {
          realisticPng: data.realistic_png,
          paperType: data.paper_type,
          inkType: data.ink_type,
          wearLevel: data.wear_level,
        },
      });
    }),
});
```

Inngest Batch Processing Function

File: `frontend/src/inngest/functions.ts`

```
import { inngest } from "./client";
import { db } from "~/server/db";

const SYNTHESIS_API_URL = process.env.SYNTHESIS_API_URL ?? "http://localhost:
8001";

export const processBatchSynthesis = inngest.createFunction(
  {
    id: "process-batch-synthesis",
    retries: 3,
    concurrency: { limit: 2, key: "event.data.userId" },
  },
  { event: "synthesis/batch.requested" },
  async ({ event, step }) => {
    const { batchJobId, userId, text, variants, generationIds } = event.data;

    // Update status to PROCESSING
    await step.run("update-status", async () => {
      await db.batchJob.update({
        where: { id: batchJobId },
        data: { status: "PROCESSING" },
      });
    });

    let successCount = 0;
```

```
// Process each variant
for (let i = 0; i < variants.length; i++) {
  const variant = variants[i];
  const generationId = generationIds[i];

  const result = await step.run(`generate-${i}`, async () => {
    const response = await fetch(`${SYNTHESIS_API_URL}/synthesize`, {
      method: "POST",
      headers: { "Content-Type": "application/json" },
      body: JSON.stringify({
        text,
        style: variant.style,
        bias: variant.bias,
        stroke_color: variant.strokeColor,
        stroke_width: variant.strokeWidth,
      }),
    });

    if (!response.ok) {
      return { success: false, error: await response.text() };
    }

    const data = await response.json();
    return { success: true, svgRaw: data.svg_raw };
  });

  // Save result
  await step.run(`save-${i}`, async () => {
    await db.savedGeneration.update({
      where: { id: generationId },
      data: {
        status: result.success ? "COMPLETED" : "FAILED",
        svgContent: result.success ? result.svgRaw : null,
        errorMessage: result.success ? null : result.error,
      },
    });

    if (result.success) {
      successCount++;
      await db.user.update({
        where: { id: userId },
        data: { credits: { decrement: 1 } },
      });
    }
  });
}

// Finalize
```

```
    await step.run("finalize", async () => {
      await db.batchJob.update({
        where: { id: batchJobId },
        data: {
          status: successCount > 0 ? "COMPLETED" : "FAILED",
          creditsUsed: successCount,
        },
      });
    });

    return { batchJobId, successCount };
  }
);
```

4.1.2 Backend Code (Python / FastAPI)

Main API Application

File: synthesis_api/main.py

```
from fastapi import FastAPI, HTTPException
from fastapi.middleware.cors import CORSMiddleware
from pydantic import BaseModel, Field, field_validator
import tempfile
import os

app = FastAPI(title="Handwriting Synthesis API")

app.add_middleware(
    CORSMiddleware,
    allow_origins=["*"],
    allow_methods=["*"],
    allow_headers=["*"],
)

VALID_CHARS = set([
    " ", "!", '"', "#", "'", "(", ")", ",", "-", ".",
    "0", "1", "2", "3", "4", "5", "6", "7", "8", "9",
    ":", ";", "?",
    "A", "B", "C", "D", "E", "F", "G", "H", "I", "J", "K", "L", "M",
    "N", "O", "P", "R", "S", "T", "U", "V", "W", "Y",
    "a", "b", "c", "d", "e", "f", "g", "h", "i", "j", "k", "l", "m",
    "n", "o", "p", "q", "r", "s", "t", "u", "v", "w", "x", "y", "z",
])

MAX_CHARS_PER_LINE = 75
MAX_LINES = 20
```

```python
# Lazy load model
_hand_instance = None

def get_hand():
    global _hand_instance
    if _hand_instance is None:
        from handwriting_synthesis import Hand
        _hand_instance = Hand()
    return _hand_instance


class SynthesisRequest(BaseModel):
    text: str = Field(..., description="Text to convert")
    style: int = Field(default=9, ge=0, le=12)
    bias: float = Field(default=0.75, ge=0.0, le=1.5)
    stroke_color: str = Field(default="black")
    stroke_width: int = Field(default=2, ge=1, le=5)

    @field_validator("text")
    @classmethod
    def validate_text(cls, v: str) -> str:
        if not v.strip():
            raise ValueError("Text cannot be empty")

        lines = v.split("\n")
        if len(lines) > MAX_LINES:
            raise ValueError(f"Maximum {MAX_LINES} lines allowed")

        for i, line in enumerate(lines):
            if len(line) > MAX_CHARS_PER_LINE:
                raise ValueError(f"Line {i+1} exceeds {MAX_CHARS_PER_LINE} ch
ars")

            invalid = [c for c in line if c not in VALID_CHARS]
            if invalid:
                raise ValueError(f"Invalid characters: {invalid}")

        return v


@app.get("/health")
async def health_check():
    return {
        "status": "healthy",
        "model_loaded": _hand_instance is not None,
        "max_chars_per_line": MAX_CHARS_PER_LINE,
        "max_lines": MAX_LINES,
    }
```

```
@app.post("/synthesize")
async def synthesize_handwriting(request: SynthesisRequest):
    try:
        hand = get_hand()
        lines = request.text.split("\n")

        with tempfile.NamedTemporaryFile(suffix=".svg", delete=False) as tmp:
            tmp_path = tmp.name

        try:
            hand.write(
                filename=tmp_path,
                lines=lines,
                biases=[request.bias] * len(lines),
                styles=[request.style] * len(lines),
                stroke_colors=[request.stroke_color] * len(lines),
                stroke_widths=[request.stroke_width] * len(lines),
            )

            with open(tmp_path, "r") as f:
                svg_content = f.read()
        finally:
            os.unlink(tmp_path)

        return {
            "svg_raw": svg_content,
            "lines_count": len(lines),
            "characters_count": sum(len(line) for line in lines),
            "style": request.style,
            "bias": request.bias,
        }

    except ValueError as e:
        raise HTTPException(status_code=400, detail=str(e))
    except Exception as e:
        raise HTTPException(status_code=500, detail=str(e))


if __name__ == "__main__":
    import uvicorn
    uvicorn.run(app, host="0.0.0.0", port=8001)
```

Post-Processing Module

File: synthesis_api/post_processing.py

```
import numpy as np
import cv2
from PIL import Image
```

```
import cairosvg
import io
import base64
from enum import Enum


class PaperType(str, Enum):
    WHITE = "white"
    CREAM = "cream"
    AGED = "aged"
    LINED = "lined"
    GRID = "grid"
    RECYCLED = "recycled"


class InkType(str, Enum):
    BALLPOINT = "ballpoint"
    GEL = "gel"
    FOUNTAIN = "fountain"
    MARKER = "marker"
    PENCIL = "pencil"


def generate_perlin_noise(width, height, scale=50.0, octaves=4):
    """Generate paper texture noise"""
    noise = np.zeros((height, width), dtype=np.float32)

    for octave in range(octaves):
        freq = 2 ** octave
        amplitude = 1 / freq

        grid_h = max(2, int(height / (scale / freq)))
        grid_w = max(2, int(width / (scale / freq)))

        grid = np.random.rand(grid_h, grid_w).astype(np.float32)
        resized = cv2.resize(grid, (width, height), interpolation=cv2.INTER_C
UBIC)
        noise += resized * amplitude

    return (noise - noise.min()) / (noise.max() - noise.min())


def generate_paper_texture(width, height, paper_type):
    """Create paper background with texture"""
    colors = {
        PaperType.WHITE: (252, 252, 250),
        PaperType.CREAM: (255, 253, 240),
        PaperType.AGED: (245, 235, 210),
        PaperType.RECYCLED: (240, 238, 230),
```

```
    }

    base_color = colors.get(paper_type, (252, 252, 250))
    img = np.full((height, width, 3), base_color, dtype=np.float32)

    # Add texture
    noise = generate_perlin_noise(width, height, scale=20.0, octaves=3)
    noise = (noise - 0.5) * 8  # Subtle variation

    for c in range(3):
        img[:, :, c] += noise

    return np.clip(img, 0, 255).astype(np.uint8)


def svg_to_alpha_mask(svg_content):
    """Convert SVG to grayscale mask"""
    png_data = cairosvg.svg2png(bytestring=svg_content.encode("utf-8"))
    img = Image.open(io.BytesIO(png_data)).convert("RGBA")

    rgb = np.array(img.convert("RGB"), dtype=np.float32)
    luminance = 0.299 * rgb[:,:,0] + 0.587 * rgb[:,:,1] + 0.114 * rgb[:,:,2]
    darkness = 255 - luminance

    alpha = np.array(img.split()[3], dtype=np.float32)
    mask = (darkness * alpha / 255).astype(np.uint8)

    return mask, img.size[0], img.size[1]


def apply_edge_roughness(mask, intensity=0.25):
    """Add micro-perturbations to edges"""
    if intensity <= 0:
        return mask

    height, width = mask.shape
    dx = generate_perlin_noise(width, height, scale=20) - 0.5
    dy = generate_perlin_noise(width, height, scale=20) - 0.5

    dx *= intensity * 2
    dy *= intensity * 2

    y, x = np.meshgrid(np.arange(height), np.arange(width), indexing="ij")
    new_x = np.clip(x + dx, 0, width - 1).astype(np.float32)
    new_y = np.clip(y + dy, 0, height - 1).astype(np.float32)

    return cv2.remap(mask.astype(np.float32), new_x, new_y,
                     interpolation=cv2.INTER_LINEAR).astype(np.uint8)
```

```python
def apply_pressure_variation(mask, intensity=0.4):
    """Modulate stroke darkness"""
    if intensity <= 0:
        return mask

    height, width = mask.shape
    noise = generate_perlin_noise(width, height, scale=80.0, octaves=2)

    min_mult = 1.0 - intensity * 0.4
    multiplier = min_mult + noise * (1.0 - min_mult)

    return (mask.astype(np.float32) * multiplier).astype(np.uint8)


def composite_ink_on_paper(ink_mask, paper, ink_color=(20, 20, 40)):
    """Blend ink onto paper"""
    height, width = ink_mask.shape
    ink_alpha = ink_mask.astype(np.float32) / 255

    result = paper.astype(np.float32)

    for c in range(3):
        result[:,:,c] = (
            result[:,:,c] * (1 - ink_alpha * 0.9) +
            ink_color[c] * ink_alpha * 0.9
        )

    return np.clip(result, 0, 255).astype(np.uint8)


def process_realistic_base64(svg_content, paper_type="white",
                             ink_type="ballpoint", wear_level=0.3,
                             stroke_color="black"):
    """Main processing function - returns base64 PNG"""

    # Convert SVG to mask
    mask, width, height = svg_to_alpha_mask(svg_content)

    # Generate paper
    paper = generate_paper_texture(width, height, PaperType(paper_type))

    # Apply effects
    mask = apply_edge_roughness(mask, intensity=0.25)
    mask = apply_pressure_variation(mask, intensity=0.4)

    # Composite
```

```
    result = composite_ink_on_paper(mask, paper)

    # Add subtle noise
    noise = np.random.randn(*result.shape) * (wear_level * 3)
    result = np.clip(result + noise, 0, 255).astype(np.uint8)

    # Convert to base64
    img = Image.fromarray(result)
    buffer = io.BytesIO()
    img.save(buffer, format="PNG")
    png_base64 = base64.b64encode(buffer.getvalue()).decode("utf-8")

    return {
        "realistic_png": png_base64,
        "width": width,
        "height": height,
        "paper_type": paper_type,
        "ink_type": ink_type,
        "wear_level": wear_level,
    }
```

4.1.3 Database Schema

File: `frontend/prisma/schema.prisma`

```
generator client {
    provider = "prisma-client-js"
    output   = "../generated/prisma"
}

datasource db {
    provider = "postgresql"
    url      = env("DATABASE_URL")
}

model User {
    id             String    @id @default(cuid())
    name           String?
    email          String?   @unique
    password       String?
    credits        Int       @default(10)

    savedGenerations SavedGeneration[]
    batchJobs        BatchJob[]
    synthesisUsage   SynthesisUsage[]

    createdAt      DateTime  @default(now())
    updatedAt      DateTime  @updatedAt
```

```
}

model SavedGeneration {
    id              String   @id @default(cuid())
    userId          String
    user            User     @relation(fields: [userId], references: [id], on
Delete: Cascade)

    status          GenerationStatus @default(COMPLETED)
    text            String   @db.Text
    style           Int
    bias            Float
    strokeColor     String   @default("black")
    strokeWidth     Int      @default(2)

    fileUrl         String?
    fileKey         String?  @unique
    svgContent      String?  @db.Text

    realisticPng    String?  @db.Text
    paperType       String?
    inkType         String?
    wearLevel       Float?

    isFavorite      Boolean  @default(false)
    tags            String[] @default([])

    batchJobId      String?
    batchJob        BatchJob? @relation(fields: [batchJobId], references: [id
])

    createdAt       DateTime @default(now())

    @@index([userId])
    @@index([status])
}

model BatchJob {
    id              String   @id @default(cuid())
    userId          String
    user            User     @relation(fields: [userId], references: [id], on
Delete: Cascade)

    name            String?
    text            String   @db.Text
    totalVariants   Int
    creditsUsed     Int      @default(0)
    status          BatchStatus @default(PENDING)
    completedCount  Int      @default(0)
```

```
    generations     SavedGeneration[]

    createdAt       DateTime @default(now())

    @@index([userId])
}

model SynthesisUsage {
    id              String   @id @default(cuid())
    userId          String
    user            User     @relation(fields: [userId], references: [id], on
Delete: Cascade)

    creditsUsed     Int
    linesCount      Int
    charactersCount Int
    style           Int
    bias            Float
    success         Boolean  @default(true)

    createdAt       DateTime @default(now())

    @@index([userId])
}

enum GenerationStatus {
    PENDING
    GENERATING
    COMPLETED
    FAILED
}

enum BatchStatus {
    PENDING
    PROCESSING
    COMPLETED
    FAILED
}
```

4.2 Data Dictionary

4.2.1 User Table

Field	Data Type	Description
id	VARCHAR	Primary key (CUID)
name	VARCHAR	User display name
email	VARCHAR	Unique email address
password	VARCHAR	Bcrypt hashed password
credits	INTEGER	Available credits (default: 10)
createdAt	TIMESTAMP	Account creation date
updatedAt	TIMESTAMP	Last modification date

4.2.2 SavedGeneration Table

Field	Data Type	Description
id	VARCHAR	Primary key (CUID)
userId	VARCHAR	Foreign key to User
status	ENUM	PENDING, GENERATING, COMPLETED, FAILED
text	TEXT	Input text content
style	INTEGER	Style index (0-12)
bias	FLOAT	Neatness value (0.0-1.5)
strokeColor	VARCHAR	CSS color value
strokeWidth	INTEGER	Stroke width (1-5)
fileUrl	VARCHAR	UploadThing URL
fileKey	VARCHAR	UploadThing file key (unique)
svgContent	TEXT	Raw SVG markup
realisticPng	TEXT	Base64 encoded PNG
paperType	VARCHAR	Paper type identifier
inkType	VARCHAR	Ink type identifier
wearLevel	FLOAT	Degradation (0.0-1.0)
isFavorite	BOOLEAN	Favorite flag
tags	TEXT[]	Array of tag strings
batchJobId	VARCHAR	Foreign key to BatchJob
createdAt	TIMESTAMP	Creation timestamp

4.2.3 BatchJob Table

Field	Data Type	Description
id	VARCHAR	Primary key (CUID)

userId	VARCHAR	Foreign key to User
name	VARCHAR	Job display name
text	TEXT	Source text
totalVariants	INTEGER	Number of variants
creditsUsed	INTEGER	Credits consumed
status	ENUM	PENDING, PROCESSING, COMPLETED, FAILED
completedCount	INTEGER	Completed count
createdAt	TIMESTAMP	Creation timestamp

4.2.4 SynthesisUsage Table

Field	**Data Type**	**Description**
id	VARCHAR	Primary key (CUID)
userId	VARCHAR	Foreign key to User
creditsUsed	INTEGER	Credits consumed
linesCount	INTEGER	Lines generated
charactersCount	INTEGER	Characters processed
style	INTEGER	Style used
bias	FLOAT	Bias value
success	BOOLEAN	Success status
createdAt	TIMESTAMP	Usage timestamp

4.3 Program Description

4.3.1 Frontend Programs

Program	File	Description
SynthesisDashboard	`synthesis-dashboard.tsx`	Main handwriting generation interface. Handles text input, style selection, bias adjustment, color picking, and displays generated SVG output.
GalleryDashboard	`gallery-dashboard.tsx`	Displays saved generations in grid view. Supports filtering, favorites, tags, and realistic rendering conversion.
BatchGenerator	`batch-generator.tsx`	Interface for generating multiple variants. Allows selecting multiple styles, tracks progress, and displays results.
Navbar	`navbar.tsx`	Navigation component with authentication state, credit balance display, and page links.

4.3.2 Backend Programs

Program	File	Description
FastAPI App	`main.py`	REST API server handling synthesis requests. Lazy-loads TensorFlow model, validates input, generates SVG.
Post-Processing	`post_processing.py`	Image processing pipeline. Converts SVG to realistic PNG with paper texture, ink effects, and artifacts.
Hand Model	`handwriting_synthesis/`	TensorFlow LSTM model for stroke generation. Uses MDN output layer for continuous coordinates.

4.3.3 tRPC Routers

Router	Procedures	Description
synthesis	generate, batchGenerate, getGallery, saveGeneration, makeRealistic	Handles all synthesis and gallery operations
credits	getBalance, purchase	Credit balance and purchase operations
auth	signIn, signUp	User authentication

4.3.4 Background Jobs

Function	Trigger Event	Description

processBatchSynthesis	`synthesis/batch.requested`	Processes batch jobs asynchronously. Iterates through variants, calls API, updates status, deducts credits.

4.4 Naming Conventions

4.4.1 File Naming

Type	Convention	Example
React Components	kebab-case.tsx	synthesis-dashboard.tsx
Python Modules	snake_case.py	post_processing.py
Prisma Schema	schema.prisma	schema.prisma
Config Files	kebab-case.js	next.config.js

4.4.2 Variable Naming

Language	Convention	Example
TypeScript Variables	camelCase	strokeColor, userId
TypeScript Constants	UPPER_SNAKE_CASE	MAX_LINES, VALID_CHARS
Python Variables	snake_case	stroke_color, user_id
Python Constants	UPPER_SNAKE_CASE	MAX_LINES, VALID_CHARS
Database Fields	camelCase	createdAt, batchJobId

4.4.3 Function Naming

Type	Convention	Example
TypeScript Functions	camelCase	handleGenerate(), validateText()
React Hooks	use prefix	useState, useSession
Python Functions	snake_case	get_hand(), process_realistic()
tRPC Procedures	camelCase verbs	getGallery, makeRealistic
API Endpoints	kebab-case	/synthesize/realistic

4.4.4 Type/Class Naming

Type	Convention	Example
TypeScript Interfaces	PascalCase	SynthesisRequest, GalleryItem
Python Classes	PascalCase	SynthesisRequest, PaperType
Enums	PascalCase	GenerationStatus, InkType
Prisma Models	PascalCase	User, SavedGeneration

4.5 Validations

Handwriting Synthesis Form:

- *Text input validation:* The synthesis form requires non-empty text input. Both client-side (TypeScript) and server-side (Python/Pydantic) validators check that the text is not empty or whitespace-only, rejecting submissions with an appropriate error message.
- *Character whitelist:* Input text is validated against a strict character whitelist containing letters (a-z, A-P, R, S, T, U, V, W, Y), digits (0-9), and common punctuation. The letters Q, X, and Z are explicitly unsupported due to training data limitations. Invalid characters trigger a validation error listing the offending characters.
- *Line and length limits:* Text is limited to a maximum of 20 lines, with each line restricted to 75 characters. The validateText() function splits input by newline and checks each line individually, returning specific error messages like "Line 3 exceeds 75 characters" if limits are exceeded.
- *Numeric parameter constraints:* Style selection uses z.number().min(0).max(12) to ensure only valid style indices (0-12) are accepted. Bias (neatness) is constrained to 0.0-1.5, and stroke width to 1-5 pixels using similar Zod schema validators.

Realistic Rendering Options:

- *Paper and ink type enums:* The paper_type field accepts only predefined values (white, cream, aged, lined, grid, recycled) validated via Python's Enum class. Similarly, ink_type is restricted to ballpoint, gel, fountain, marker, or pencil.
- *Wear level bounds:* The wear_level parameter uses Pydantic's Field(ge=0.0, le=1.0) constraint to ensure the degradation intensity stays within the valid 0-1 range, where 0 represents pristine output and 1 represents heavily worn appearance.

Gallery and Storage:

- *File upload validation:* UploadThing configuration restricts uploads to SVG files (image/svg+xml MIME type) with a maximum size of 4MB. The middleware verifies user authentication before allowing uploads.
- *Ownership verification:* All gallery operations (view, edit, delete, make realistic) include a userId filter in database queries to ensure users can only access their own generations. Attempting to access another user's resource returns a NOT_FOUND error rather than FORBIDDEN, preventing resource enumeration attacks.
- *Unique constraints:* Database-level unique constraints on fileKey and realisticKey prevent duplicate file references. The email field on User is also unique to prevent duplicate accounts.

Authentication and Authorization:

- *Session validation:* Protected tRPC procedures use middleware that checks for a valid session. If ctx.session or ctx.session.user is null/undefined, the request is rejected with an UNAUTHORIZED error code before any business logic executes.
- *Credit verification:* Before any credit-consuming operation (synthesis, batch generation), the system queries the user's current credit balance. If credits are insufficient for the requested operation, a FORBIDDEN error is thrown with the message "Insufficient credits" and the operation is aborted.

Batch Processing:

- *Variant limits:* Batch generation requests are limited to 1-13 variants maximum using z.array(…).min(1).max(13), corresponding to one variant per available style.
- *Credit pre-check:* Before queuing a batch job, the system verifies the user has enough credits for all requested variants. The check compares user.credits against variants.length to prevent jobs from starting that cannot complete.

API Route Usage: These validation schemas are applied consistently across the stack. The tRPC router parses and validates input against Zod schemas, throwing TRPCError with appropriate codes (BAD_REQUEST for validation failures, FORBIDDEN for authorization failures). The Python FastAPI backend uses Pydantic models with field_validator decorators that raise ValueError exceptions, automatically converted to 400 Bad Request